

The JSON Best Practices Cheat Sheet

1. Naming Conventions

Property Names

- **Use camelCase** (e.g., `userProfile`, `createdAt`)
- This is the standard for JavaScript/JSON environments
- Improves readability and consistency across your codebase

✗ Avoid

- **Hyphens:** `user-profile` (causes parsing errors in some languages)
- **Spaces:** `user profile` (invalid JSON syntax)
- **Special characters:** `user$profile`, `user@name`

Example:

```
JSON
{
  "userId": 12345,
  "firstName": "Ahmed",
  "lastName": "Rahman",
  "createdAt": "2025-01-05T10:00:00Z",
  "isActive": true
}
```

2. Data Types Do's & Don'ts

Dates

Always use **ISO 8601** format for universal compatibility:

✓ **Good:** `"2025-12-29T10:00:00Z"`

- Clear timezone (Z = UTC)
- Sortable
- Widely supported

✗ Bad:

- "12/29/2025" (Ambiguous - US vs UK format)
- "29-12-2025" (Not machine-readable)
- 1735466400 (Unix timestamp lacks human readability)

Null Values

- Use `null` for truly missing values
- Don't use empty strings "" when you mean "no value"

Example:

```
JSON
{
  "name": "John Doe",
  "middleName": null,
  "email": "john@example.com",
  "phoneNumber": null
}
```

Numbers

- Don't wrap numbers in quotes unless necessary
- Exception: Very large integers (>15 digits) to prevent precision loss

✓ Good:

```
JSON
{
  "price": 29.99,
  "quantity": 5
}
```

✗ Bad:

```
JSON
{
  "price": "29.99",
  "quantity": "5"
}
```

3. Common Validation Errors

Trailing Commas

 **Not Allowed in Standard JSON:**

```
JSON
{
  "name": "John",
  "age": 30, ← This comma breaks standard JSON
}
```

 **Correct:**

```
JSON
{
  "name": "John",
  "age": 30
}
```

Quote Style

- Keys **must** be wrapped in **double quotes** "key"
- Single quotes 'key' are **not valid** JSON

 **Wrong:**

```
JSON
{
  'name': 'John',
  age: 30
}
```

 **Correct:**

```
JSON
{
  "name": "John",
  "age": 30
}
```

```
}
```

Common Syntax Mistakes

1. **Forgetting quotes on keys**
 2. **Using single quotes instead of double**
 3. **Adding trailing commas**
 4. **Not escaping special characters**
 5. **Mixing data types** (e.g., `"true"` vs `true`)
-

4. Structure Best Practices

Keep It Flat When Possible

 **Over-nested:**

JSON

```
{
  "user": {
    "profile": {
      "name": {
        "first": "John",
        "last": "Doe"
      }
    }
  }
}
```

 **Better:**

JSON

```
{
  "firstName": "John",
  "lastName": "Doe"
}
```

Use Arrays for Lists

```
JSON
{
  "users": [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"}
  ]
}
```

Consistent Ordering

Keep keys in a logical order:

1. ID fields first
 2. Name/title fields
 3. Data fields
 4. Metadata (timestamps, status)
-

5. Security Considerations

Never Include Sensitive Data

❌ Don't expose:

- Passwords (even hashed)
- API keys
- Credit card numbers
- Social security numbers

Validate Before Trusting

Always validate JSON from external sources:

- Check for expected keys
 - Validate data types
 - Sanitize string values
 - Implement maximum size limits
-

Quick Reference Card

Issue	Problem	Solution
Trailing comma	<code>{"a":1, }</code>	Remove last comma
Single quotes	<code>{'a':1}</code>	Use double quotes
Unquoted keys	<code>{a:1}</code>	Quote all keys
Wrong date	<code>"12/29/25"</code>	Use ISO 8601
Wrong null	<code>" "</code>	Use <code>null</code>

Tools & Libraries

JavaScript:

- Native: `JSON.parse()`, `JSON.stringify()`
- Validation: `ajv`, `joi`

Python:

- Native: `json` module
- Validation: `jsonschema`

PHP:

- `json_encode()`, `json_decode()`
- Always check `json_last_error()`

Need Help?

Need a robust API architecture?

Devxhub specializes in building scalable, secure backend systems with proper data validation and error handling.

 Contact us: info@devxhub.com

 Visit: www.devxhub.com

 Call: +880 1326 506464
